

MANUALE TECNICO – PROGETTO THE KNIFE

Versione: 1.0 | Data: 11/09/2026

1. Introduzione ed Obiettivi del Sistema

Il progetto *The Knife* nasce per rispondere alla crescente esigenza di digitalizzazione, automazione e gestione centralizzata nel settore della ristorazione moderna. L'applicazione si configura come una soluzione software distribuita, progettata per consentire l'interazione in tempo reale tra tre macrocategorie di attori: i clienti finali (alla ricerca di locali, recensioni e servizi di prenotazione e takeaway), i gestori dei ristoranti (responsabili della manutenzione del proprio catalogo e del monitoraggio dei flussi di prenotazione) e gli amministratori di sistema (incaricati della supervisione globale della piattaforma).

L'obiettivo primario dell'architettura software è duplice. Da un lato, garantire un'esperienza utente (UX) fluida, reattiva e priva di latenze percettibili attraverso l'impiego del framework grafico JavaFX. Dall'altro, assicurare un livello elevatissimo di integrità, sicurezza, consistenza e isolamento dei dati sul server, gestendo in modo ottimale la concorrenza multiutente tipica dei sistemi distribuiti ad alto traffico.

1.1 Contesto Applicativo e Requisiti di Business

Nel panorama attuale, la ristorazione richiede strumenti agili per la gestione delle prenotazioni e la fidelizzazione della clientela. *The Knife* affronta questa problematica offrendo un database relazionale centralizzato (PostgreSQL) accessibile esclusivamente tramite un server logico intermedio. Questo approccio impedisce ai client periferici di manipolare direttamente la base di dati, riducendo drasticamente i rischi di corruzione dei dati o di vulnerabilità legata a iniezioni SQL dirette sul client.

2. Architettura del Sistema e Scelte Progettuali

L'architettura software di *The Knife* è basata su un modello **Client-Server distribuito a due livelli**, accoppiato a una rigorosa separazione delle responsabilità.

2.1 Analisi del Tier di Presentazione (Client)

Il Client è l'applicazione front-end con cui l'utente interagisce direttamente. È stato sviluppato in **Java** utilizzando il framework **JavaFX**, che permette di definire interfacce grafiche moderne, scalabili e basate su file di markup descrittivi (.fxml) separati dalla logica di controllo (Controller).

- **Leggerezza del Client:** Il client non contiene logica di business complessa né driver di connessione diretti al database relazionale. Il suo compito principale consiste nel raccogliere l'input dell'utente, effettuare validazioni formali di primo livello (es. controllo campi vuoti o formati errati) e serializzare le richieste da inviare al server.
- **Reattività della GUI:** Grazie all'uso dei thread di JavaFX e dei listener reattivi (Observer Pattern), l'interfaccia grafica si aggiorna dinamicamente in risposta agli eventi generati dall'utente o alle notifiche di ritorno dal server.

2.2 Analisi del Tier di Logica e Persistenza (Server)

Il Server rappresenta il cuore pulsante dell'applicazione. Esso agisce come un demone centralizzato in ascolto su una porta di rete dedicata, gestendo le richieste provenienti da molteplici client distribuiti.

- **Logica Applicativa Centralizzata:** Tutte le regole di business (calcolo delle medie delle recensioni, autorizzazioni basate sui ruoli) risiedono esclusivamente sul server. Questo garantisce che eventuali aggiornamenti logici non richiedano la ridistribuzione del software client a tutti gli utenti.
- **Gestione della Concorrenza:** Il server è progettato per essere *thread-safe*. Ogni chiamata remota viene eseguita in contesti di esecuzione isolati, prevenendo condizioni di stallo (*deadlock*) o race condition durante la scrittura simultanea di prenotazioni sullo stesso ristorante.

3. Il Protocollo Java RMI (Remote Method Invocation)

Per la comunicazione tra il Client e il Server, il progetto adotta la tecnologia **Java RMI**. Questa scelta tecnologica merita un approfondimento ingegneristico per comprendere i vantaggi rispetto ad altre soluzioni di mercato, come le API REST basate su HTTP/JSON.

3.1 Motivazioni della Scelta di Java RMI

1. **Trasparenza della Chiamata Remota:** RMI consente al codice client di invocare un metodo su un oggetto residente sul server esattamente come se quell'oggetto fosse allocato localmente nella memoria heap del client. L'infrastruttura RMI si occupa autonomamente del marshalling (serializzazione degli oggetti in byte) e dell'unmarshalling (ricostruzione degli oggetti all'arrivo).
2. **Type-Safety e Condivisione dei Modelli:** Poiché l'intera applicazione è scritta in Java, le classi del package model (come Ristorante, Utente, Prenotazione) sono condivise identiche tra client e server. Questo elimina la necessità di scrivere costosi mapper o logiche di conversione da JSON a oggetti Java tipizzati.
3. **Contratti Rigidi tramite Interfacce:** L'architettura RMI impone l'uso di interfacce estendenti `java.rmi.Remote` i cui metodi sollevano `java.rmi.RemoteException`. Questo costringe il programmatore a gestire esplicitamente i fallimenti di rete, aumentando la robustezza complessiva del sistema distribuito.

4. Modellazione Dati e Schema Entità-Relazione (ER)

La persistenza delle informazioni del sistema *The Knife* è affidata a un database relazionale gestito tramite **PostgreSQL**. La progettazione della base di dati è partita da un'analisi concettuale approfondita, tradotta successivamente in un modello logico ottimizzato per le prestazioni di lettura e scrittura.

4.1 Analisi delle Entità Principali

Il dominio applicativo è strutturato attorno a quattro entità fondamentali, ciascuna dotata di attributi specifici e vincoli di integrità:

1. **Utente:** Rappresenta qualsiasi soggetto registrato sulla piattaforma. Gli attributi principali includono l'identificativo univoco (id), nome, cognome, username (utilizzato come chiave alternativa per il login), la password, il ruolo di appartenenza e altri attributi opzionali che lo descrivono. Dal diagramma ER si evitano le seguenti interazioni strutturali:
 - Gestione Ristoranti: È collegato tramite la relazione gestisce con cardinalità (0,N) per l'utente e (1,1) per il ristorante (un utente gestore può gestire zero o più ristoranti, mentre un ristorante è gestito esattamente da un utente).

- Prenotazioni: L'utente effettua prenotazioni con cardinalità (0,N) lato utente e (1,1) lato prenotazione.
- Preferiti: Tramite la relazione ha, l'utente intrattiene un legame (0,N) con i ristoranti preferiti.
- Recensioni: L'utente scrive recensioni con cardinalità (0,N) lato utente e (1,1) lato recensione.

2. **Ristorante:** Modella l'esercizio commerciale all'interno della piattaforma. Contiene informazioni quali id, nome, indirizzo, città, tipologia di cucina, fascia di prezzo e altri attributi che lo descrivono (servizi, premi, takeaway, prenotazioni, ...). Le relazioni modellate nel diagramma comprendono:

- Gestione: Collegato all'utente tramite la relazione gestisce con cardinalità (1,1).
- Prenotazioni: L'entità controlla le prenotazioni con cardinalità (1,1) sul lato prenotazione e (0,N) sul lato ristorante.
- Preferiti: È associato tramite la relazione è con cardinalità (1,1) rispetto all'entità Preferito.
- Recensioni: Il ristorante riceve recensioni con cardinalità (1,1) sul lato recensione e (0,N) sul lato ristorante.

3. **Prenotazione:** Gestisce l'interazione temporale tra il cliente e il ristorante. Include id, data, ora, numero di coperti e lo stato corrente (es. In Attesa, Confermata, Rifiutata). Dal diagramma ER, la prenotazione è vincolata da relazioni con cardinalità (1,1) sia verso il ristorante (tramite la relazione controlla) sia verso l'utente (tramite la relazione effettua), garantendo che ogni prenotazione faccia capo a un unico utente e a un unico ristorante.

4. **Recensione:** Permette ai clienti di esprimere un giudizio valutativo sui locali visitati. Comprende id, punteggio numerico, testo descrittivo e data. Le relazioni evidenziate nel diagramma sono:

- Scrittura: Collegata all'utente tramite la relazione scrive con cardinalità (1,1).

- **Ricezione:** Collegata al ristorante tramite la relazione riceve con cardinalità (1,1).
 - **Risposta:** È associata all'entità RispostaRecensione tramite la relazione risponde con cardinalità (0,1) per la recensione e (1,1) per la risposta, indicando che una recensione può ricevere al massimo una risposta da parte del gestore.
5. **Preferito:** Entità di associazione che modella i ristoranti salvati dagli utenti. Presenta cardinalità (1,1) sia verso il ristorante (relazione è) sia verso l'utente (relazione ha), fungendo da tabella di giunzione per la gestione della lista dei preferiti.
6. **RispostaRecensione:** Modella l'eventuale replica scritta dal gestore a una recensione ricevuta. Si relaziona con l'entità Recensione attraverso la relazione risponde con cardinalità (1,1) per la risposta e (0,1) per la recensione, riflettendo la possibilità opzionale di risposta da parte del locale.

4.2 Schema ER e Vincoli d'Integrità

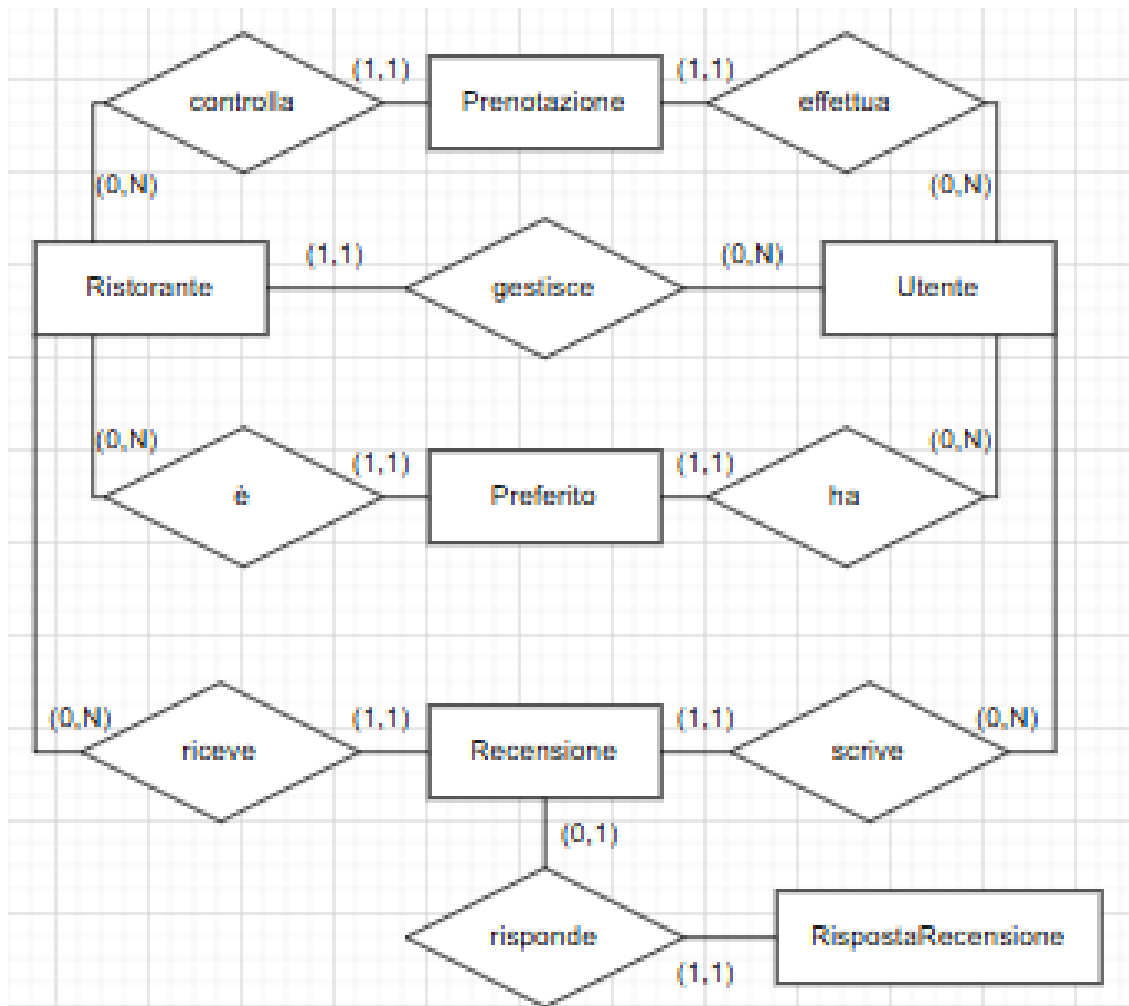


Figura 1: Schema Entità-Relazione (ER) del database The Knife. Il diagramma evidenzia le sei entità principali (Utente, Ristorante, preferito, Prenotazione, Recensione, RispostaRecensione) e le relative cardinalità che legano.

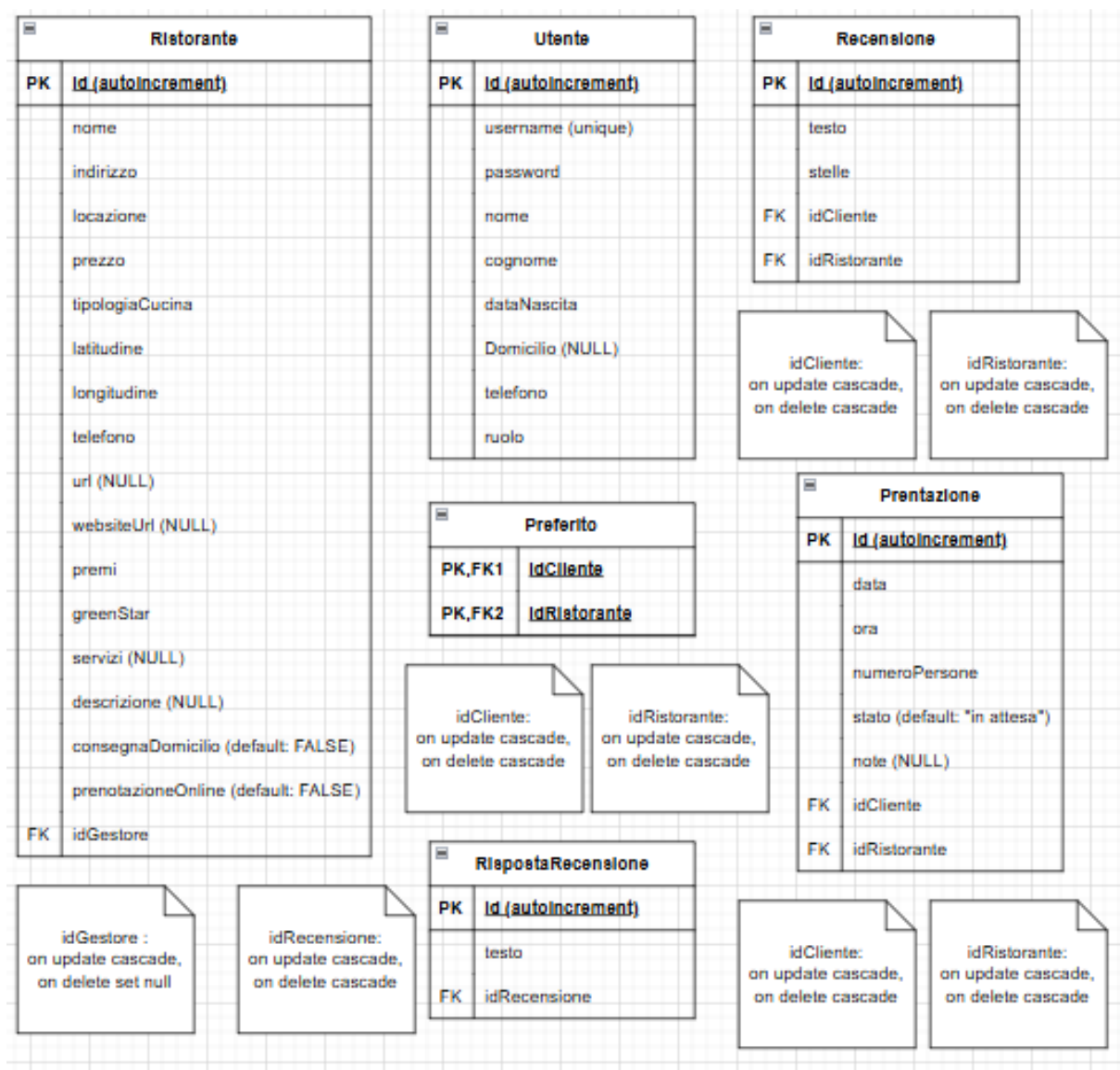


Figura 2: Schema Logico e Relazionale del database The Knife. Il diagramma illustra gli attributi dell'entità e la struttura tabellare implementata su PostgreSQL, evidenziando le chiavi primarie (PK), le chiavi esterne (FK) e le relative regole di integrità referenziale (on update cascade, on delete cascade / on set null) che collegano le tabelle Ristorante, Utente, Prenotazione, Recensione, Preferito e RispostaRecensione.

5. Il Processo di Normalizzazione (Terza Forma Normale - 3NF)

Per prevenire anomalie di inserimento, cancellazione e aggiornamento all'interno del database PostgreSQL, lo schema relazionale è stato sottoposto a un iter formale di normalizzazione, portandolo fino alla **Terza Forma Normale (3NF)**.

5.1 Prima Forma Normale (1NF)

Una relazione è in 1NF se tutti i suoi attributi contengono valori atomici (non scomponibili) e i domini di tali attributi contengono solo valori scalari.

- *Applicazione nel progetto:* Nello schema iniziale, si è evitato di inserire campi multivalore (come liste di numeri telefonici o array di recensioni all'interno della tabella del ristorante). Ogni cella della tabella interseca una riga e una colonna con un valore singolo, eliminando i gruppi ripetuti.

5.2 Seconda Forma Normale (2NF)

Una relazione si trova in 2NF se è in 1NF e tutti gli attributi non chiave dipendono funzionalmente e interamente dall'intera chiave primaria (non esistono dipendenze parziali da sottoinsiemi di chiavi composte).

- *Applicazione nel progetto:* Nelle tabelle con chiavi primarie semplici (composte da un unico attributo identificativo autoincrementale come idristorante o idutente), la condizione di 2NF è soddisfatta nativamente. Per le tabelle di associazione, si è fatto ricorso a chiavi primarie surrogate per evitare chiavi composte complesse e dipendenze parziali.

5.3 Terza Forma Normale (3NF)

Una relazione è in 3NF se è in 2NF e non esistono dipendenze transitive tra gli attributi non chiave (ovvero, nessun attributo non chiave dipende da un altro attributo non chiave).

- *Applicazione nel progetto:* Un esempio emblematico riguarda la gestione del gestore di un ristorante. In una progettazione errata non normalizzata, i dati anagrafici del gestore (nome, username, telefono, ...) avrebbero potuto essere duplicati direttamente dentro la tabella Ristorante. Questo avrebbe generato un'anomalia di aggiornamento nel caso in cui il gestore avesse cambiato username (obbligando a modificare tutte le righe dei ristoranti a lui associati).
- *Soluzione adottata:* La tabella Ristorante contiene esclusivamente la chiave esterna idgestore. I dati anagrafici risiedono unicamente nella tabella Utente. Modificando l'utente, la modifica si riflette immediatamente su tutte le query che coinvolgono il ristorante, garantendo la massima consistenza transazionale.

6. Struttura del Software e Diagrammi UML

L'architettura statica e dinamica del codice sorgente di *The Knife* rispecchia i principi dell'ingegneria del software orientata agli oggetti. L'adozione di un design pulito garantisce un basso accoppiamento (*low coupling*) e un'alta coesione (*high cohesion*) tra i moduli.

6.1 Diagramma delle Classi UML

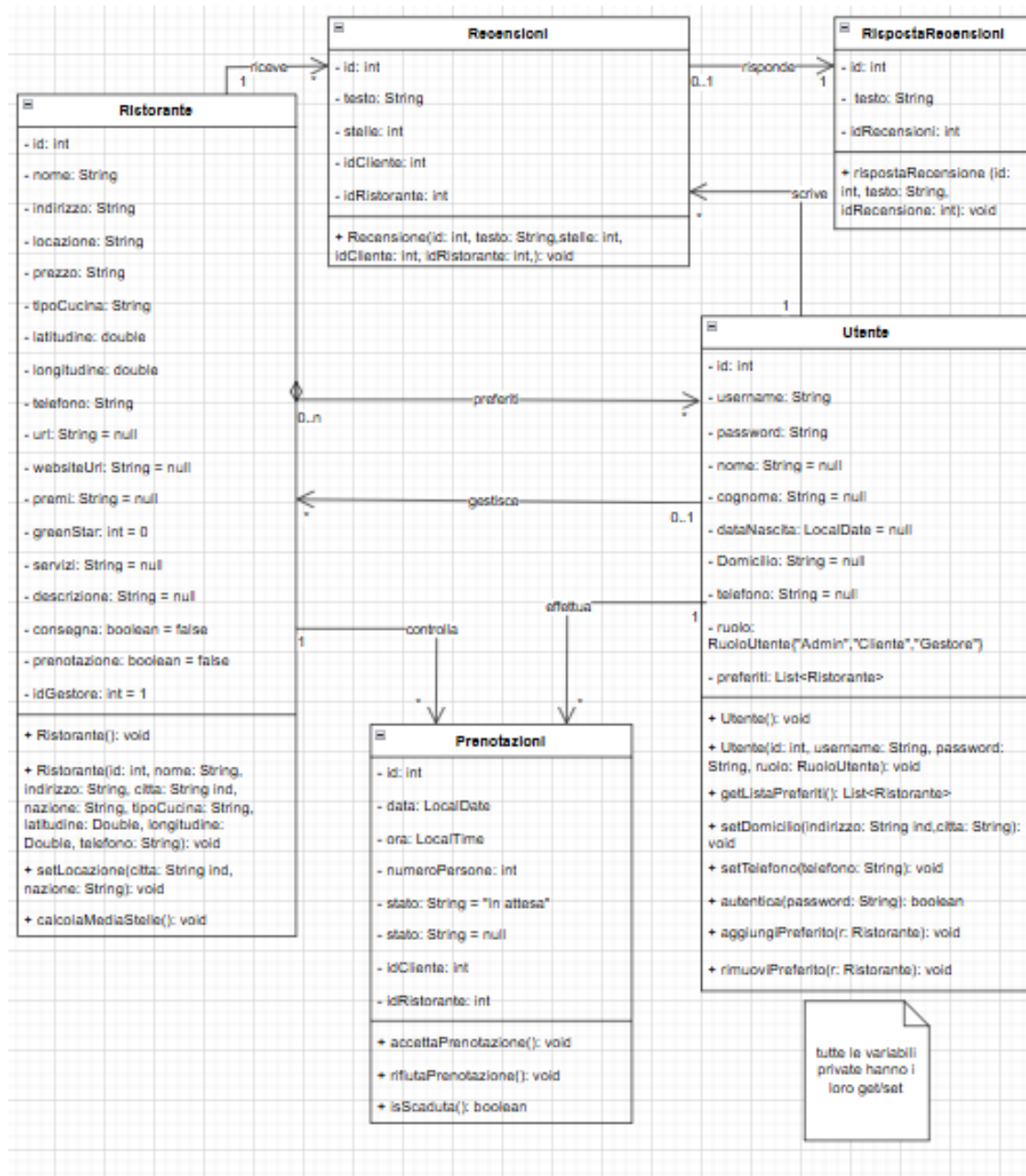


Figura 3: Diagramma delle Classi UML del sistema *The Knife*. Il diagramma illustra la struttura a oggetti del dominio applicativo, evidenziando le classi principali (*Ristorante*, *Utente*, *Recensioni*, *RispostaRecensioni*, *Prenotazioni*), i relativi attributi tipati, le operazioni supportate (inclusi i metodi di gestione e

calcolo come `calcolaMediaStelle()`) e le associazioni con le rispettive cardinalità che legano i componenti del sistema.

- **Analisi della struttura statica:** Il diagramma evidenzia la netta separazione dei package all'interno dell'applicazione:
 - **Package `client.view` e `client.controller`:** Racchiudono le componenti grafiche JavaFX e i relativi controller che gestiscono gli eventi generati dall'utente sulla GUI.
 - **Package `server.service`:** Implementa la logica applicativa remota, esponendo i metodi attraverso le interfacce RMI.
 - **Package `server.dao`:** Raggruppa le classi responsabili dell'interazione diretta con il database PostgreSQL, isolando completamente i comandi SQL dal resto del sistema.

6.2 Diagramma di Sequenza UML

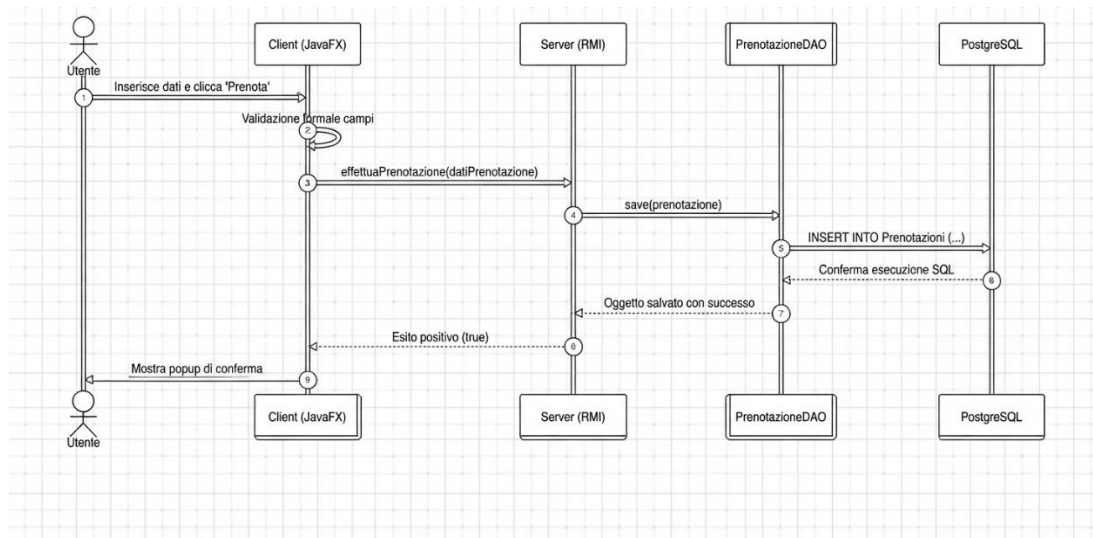


Figura 4: Diagramma di Sequenza UML (Flusso di Prenotazione). Il diagramma illustra l'interazione dinamica tra le componenti del sistema durante il processo di prenotazione di un tavolo. Il flusso descrive la catena di chiamate che, partendo dall'interazione dell'utente con il client JavaFX, propaga la richiesta attraverso l'infrastruttura RMI verso il server, delegando al DAO l'interazione persistente con il database PostgreSQL per la finalizzazione dell'operazione.

- *Analisi della dinamica transazionale:* Il diagramma di sequenza modella il flusso di esecuzione tipico di un'operazione complessa, come la registrazione di una nuova prenotazione:
 1. L'utente interagisce con il form JavaFX sul Client.
 2. Il Controller del Client invoca il metodo remoto esposto dal Server tramite l'interfaccia RMI.
 3. Il Server riceve la chiamata, convalida i parametri di business e interPELLa la classe DAO competente.
 4. Il DAO esegue la query SQL transazionale sul database PostgreSQL sfruttando il connection pool.
 5. L'esito viene propagato a ritroso attraverso la catena RMI, aggiornando in tempo reale la visualizzazione grafica sul Client.

7. Design Pattern Applicati

Per risolvere problemi ricorrenti di progettazione e garantire la manutenibilità del codice nel lungo periodo, sono stati implementati diversi Design Pattern consolidati:

7.1 Data Access Object (DAO) Pattern

Il pattern DAO costituisce il pilastro dello strato di persistenza. Le classi dedicate (RistoranteDAO, UtenteDAO, PrenotazioneDAO, ...) disaccoppiano totalmente la logica di business dai dettagli motoristici del database relazionale.

- *Vantaggio ingegneristico:* Se in futuro l'applicazione dovesse migrare da PostgreSQL a un differente DBMS (es. MySQL o Oracle), l'intervento di refactoring sarebbe circoscritto esclusivamente alle classi DAO, lasciando inalterati i servizi di business e l'interfaccia grafica.

7.2 Singleton Pattern

Il pattern Singleton è impiegato nella gestione delle configurazioni globali e nella connessione al database.

- *Vantaggio ingegneristico:* Garantisce l'esistenza di un'unica istanza del gestore delle risorse di connessione durante tutto il ciclo di vita del server, prevenendo il consumo

anomalo di memoria e la creazione incontrollata di descrittori di rete verso PostgreSQL.

7.3 MVC (Model-View-Controller) e Observer Pattern

- **MVC in JavaFX:** La separazione tra la struttura visiva (definita nei file descrittivi .fxml), la logica di controllo (Controller) e i dati strutturati (Model) permette a più sviluppatori di lavorare in parallelo sull'interfaccia e sul motore senza conflitti di merge.
- **Observer Pattern:** Sfruttato intensamente all'interno dei componenti JavaFX (tramite listener sugli eventi di click o proprietà reattive) per aggiornare automaticamente le tabelle e i pannelli informativi non appena un evento asincrono viene completato dal server.

8. Scelte di Programmazione e Ottimizzazione del Server

Nello sviluppo del backend di *The Knife*, particolare attenzione è stata posta alle performance di I/O e alla gestione delle risorse di rete. L'apertura e la chiusura continua di connessioni JDBC verso PostgreSQL rappresentano un collo di bottiglia computazionale critico per qualsiasi applicazione distribuita.

8.1 Gestione del Connection Pooling con HikariCP

Per superare la latenza associata alla creazione dinamica delle connessioni al database, il server integra **HikariCP**, un framework di connection pooling ad altissime performance, leggero e thread-safe.

- **Snippet di Configurazione:**

Java

```
HikariConfig config = new HikariConfig();  
  
config.setJdbcUrl("jdbc:postgresql://localhost:5432/TheKnifeVirgilio");  
  
config.setUsername("tuo_utente");  
  
config.setPassword("tua_password");  
  
config.setMaximumPoolSize(10);
```

```
config.setMinimumIdle(5);
```

```
config.setIdleTimeout(30000);
```

```
config.setConnectionTimeout(30000);
```

```
HikariDataSource dataSource = new HikariDataSource(config);
```

- *Analisi dei parametri:*
 - `setMaximumPoolSize(10)` imposta il numero massimo di connessioni fisiche simultanee che il pool può aprire verso PostgreSQL, prevenendo la saturazione delle risorse del DBMS.
 - `setMinimumIdle(5)` garantisce che vi siano sempre almeno 5 connessioni pronte all'uso, azzerando i tempi di attesa per le prime richieste dei client.
 - `setConnectionTimeout(30000)` definisce il tempo massimo (in millisecondi) che un thread deve attendere prima di ricevere una connessione libera dal pool, sollevando un'eccezione controllata in caso di congestione prolungata.

8.2 Gestione degli Errori e Resilienza RMI

Nelle comunicazioni di rete distribuite, i fallimenti (come la caduta temporanea della connessione o il riavvio del server) sono eventi da mettere in conto. Le classi del client implementano blocchi try-catch mirati a intercettare le eccezioni di tipo `RemoteException`.

- *Strategia di recupero:* In caso di interruzione della comunicazione, l'interfaccia grafica JavaFX intercetta l'eccezione e mostra un messaggio di errore descrittivo all'utente (es. *"Impossibile contattare il server remoto. Riprovare più tardi"*), impedendo il crash inatteso dell'applicazione client.

9. Testing, Documentazione Javadoc e Manutenzione

Il ciclo di vita del software di *The Knife* comprende rigide pratiche di verifica della qualità del codice e documentazione tecnica automatizzata.

9.1 Documentazione con Standard Javadoc

Il codice sorgente dell'intero progetto è commentato capillarmente utilizzando i tag standard di **Javadoc**.

- Ogni classe pubblica, interfaccia remota e metodo di servizio è corredato da blocchi di documentazione formale che specificano:
 - `@param`: Descrizione dettagliata dei parametri in input.
 - `@return`: Valore restituito dal metodo e relative precondizioni.
 - `@throws`: Eccezioni sollevate in condizioni anomale (es. `SQLException` o `RemoteException`).
- *Generazione della documentazione*: Tramite il comando di build di Maven (`mvn javadoc:javadoc`), il sistema compila automaticamente la documentazione in un set completo di pagine HTML consultabili offline nella cartella `/target/site/apidocs/index.html`.

9.2 Politiche di Testing e Validazione

Il software è stato sottoposto a test incrementali nelle diverse fasi di sviluppo:

1. **Unit Testing**: Le classi DAO e i componenti di calcolo della logica di business sono stati verificati isolatamente per confermare la correttezza delle query SQL e la manipolazione dei dati.
2. **Integration Testing**: L'infrastruttura RMI è stata testata avviando il server e simulando richieste concorrenti multiple da parte di client distinti, verificando la stabilità del connection pool di HikariCP sotto carico.
3. **User Acceptance Testing (UAT)**: Le viste JavaFX sono state validate sul campo per accertare la coerenza dei flussi operativi sia per il ruolo Cliente che per il ruolo Gestore.

10. Interfaccia Grafica e Risultati Finali

L'interfaccia utente, sviluppata interamente in JavaFX, garantisce un'esperienza coerente, moderna e reattiva per tutte le tipologie di utenti. Di seguito vengono presentate le schermate chiave del sistema.

10.1 Schermata Iniziale e Autenticazione (Login / Registrazione)

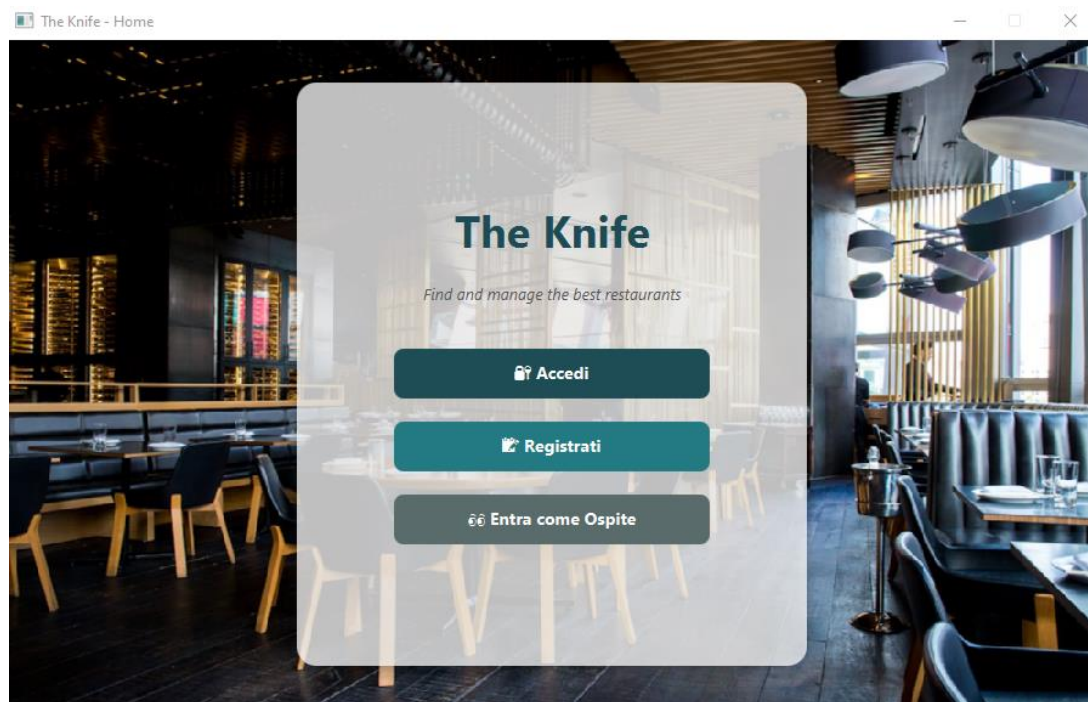


Figura 5.1: La schermata di benvenuto (HomeView)

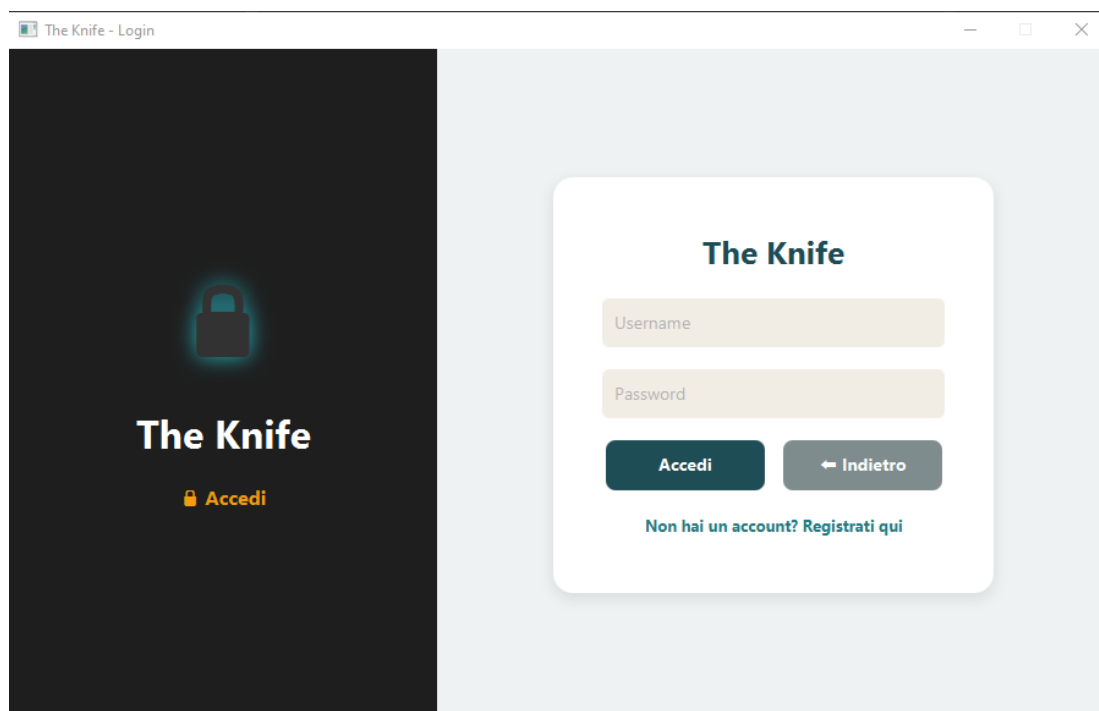


Figura 5.2: La schermata di login (loginView)

The Knife - Registrazione

Crea un nuovo account

Nome * Cognome *

Username *

Password *

Data di Nascita  Indirizzo

Città CAP Nazione

Telefono (Opzionale)

Cliente Standard ▼

Registrati ← Indietro

[Hai già un account? Accedi qui](#)

Figura 5.3: La schermata di registrazione (SigninView)

- Commento all'immagine: La schermata di benvenuto (*Figura 5.1*) consente l'accesso alle funzionalità principali offrendo tre modalità distinte: autenticazione, registrazione o accesso come ospite. La procedura di registrazione (*Figura 5.3*) e il login (*Figura 5.2*) prevedono una validazione preventiva dei campi lato client prima di inoltrare la richiesta al server tramite RMI.

10.2 Dashboard di Ricerca e Catalogo Ristoranti

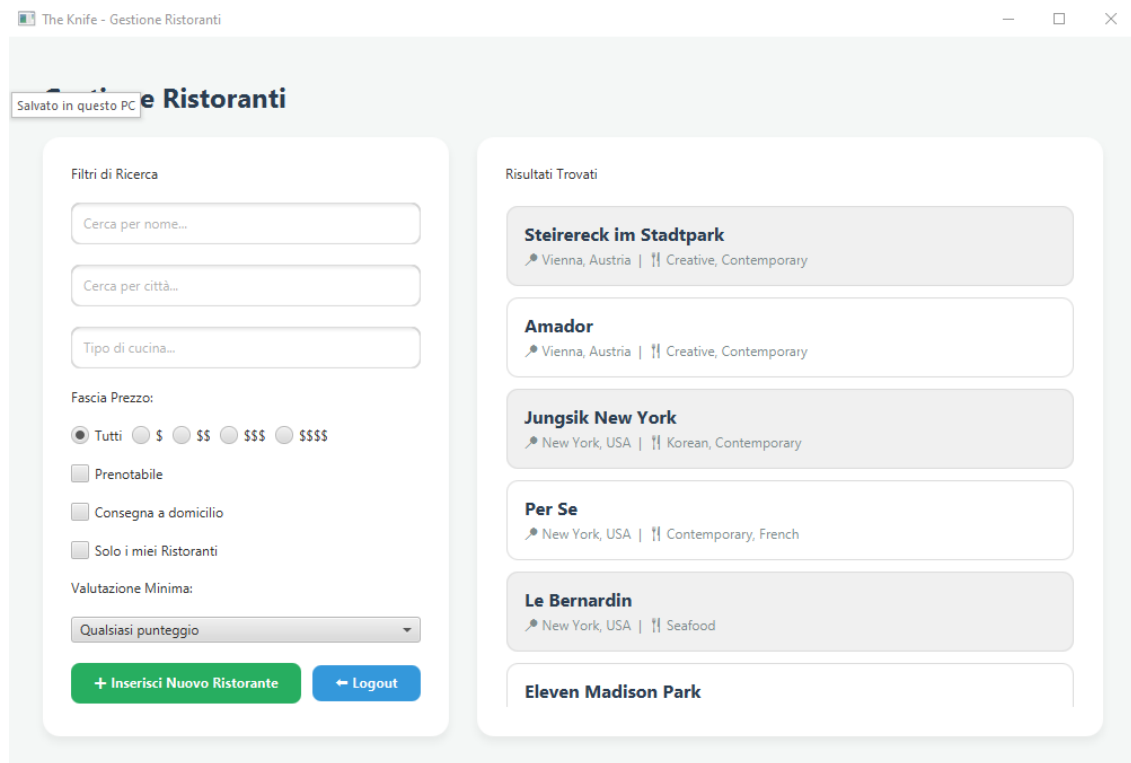


Figura 5.4: Elenco ristoranti (*CercaRistoranteGestoreView*)

- Commento all'immagine: Il pannello principale per la gestione e consultazione dei ristoranti (*Figura 5.4*) mostra l'elenco dei locali disponibili. Sulla sinistra, i filtri avanzati (ricerca per nome, città, tipo di cucina, fascia di prezzo e opzioni di servizio) permettono di raffinare i risultati in tempo reale, interagendo dinamicamente con il server e il database.
- Sebbene le funzionalità di ricerca siano strutturalmente e graficamente analoghe, il sistema distingue internamente tra i metodi *CercaRistoranteAnonimoView* e *CercaRistoranteClienteView*. Entrambi operano sulla medesima logica di interrogazione al database, ma si differenziano per le opzioni di scelta disponibili: mentre l'interfaccia per l'utente non autenticato offre filtri generici, quella per l'utente loggato arricchisce le possibilità di ricerca includendo opzioni personalizzate, come ad esempio la visualizzazione filtrata dei soli ristoranti preferiti o gestiti direttamente dall'utente, garantendo un'esperienza di navigazione più profilata e contestualizzata. Inoltre, all'interno di questa stessa pagina, l'interfaccia si adatta dinamicamente offrendo bottoni dedicati in base al ruolo: il gestore può sfruttare il comando apposito per aggiungere un nuovo ristorante al sistema, mentre il cliente ha la possibilità di accedere rapidamente alla visualizzazione delle proprie prenotazioni cliccando sul rispettivo pulsante di collegamento.

10.3 Dettaglio Ristorante e Gestione Recensioni

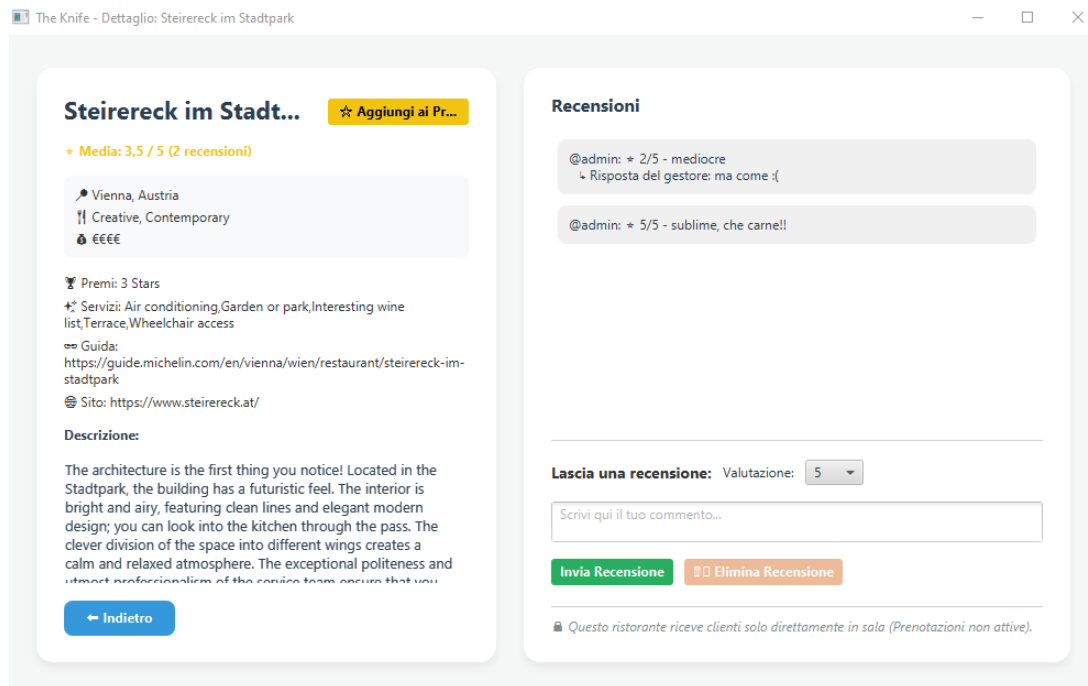


Figura 5.5: Dettaglio ristoranti (DettaglioRistoranteGestoreView)

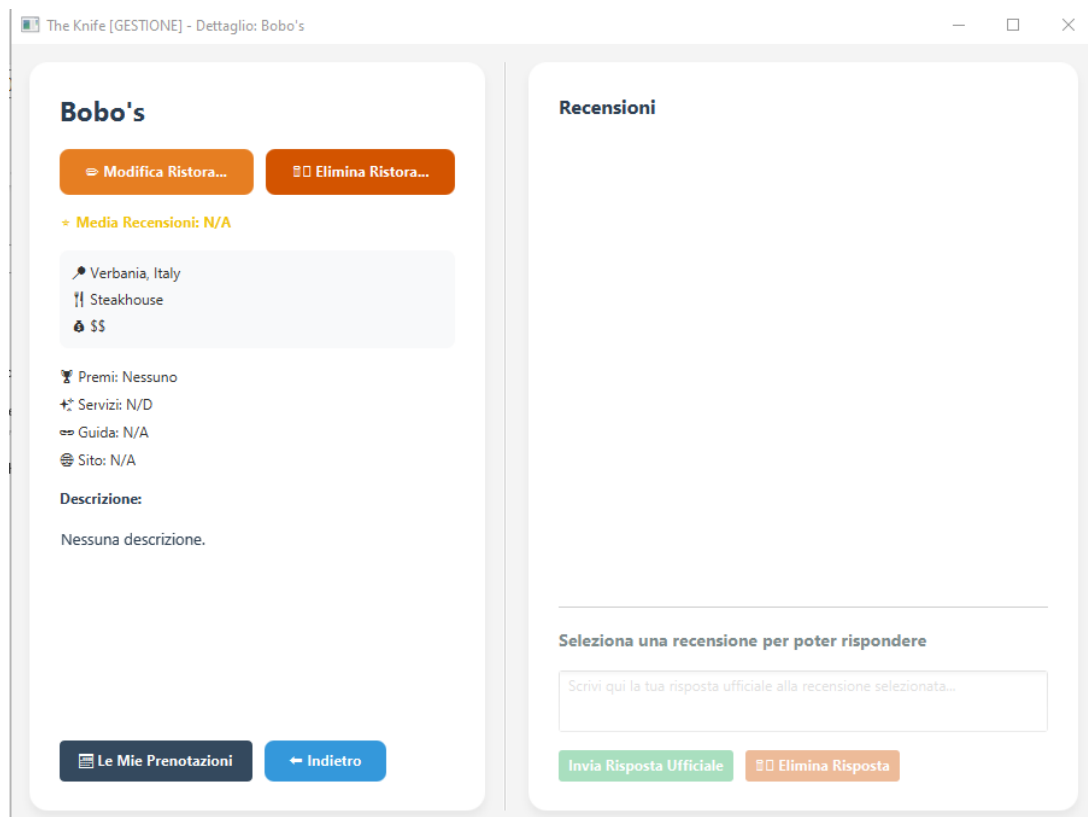


Figura 5.6: Dettaglio ristoranti (DettaglioRistoranteClienteView)

- Commento all'immagine: La vista di dettaglio del singolo ristorante (*Figura 5.5*) mostra le informazioni complete (premi, servizi, guida Michelin e descrizione) insieme alla sezione delle recensioni degli utenti. Da questa schermata è possibile calcolare la media dei punteggi e inserire un nuovo commento o valutazione.
- Analogamente alla logica di ricerca, anche le funzionalità di interazione con il dettaglio del ristorante variano in base al ruolo dell'utente autenticato: l'utente ospite gode di un accesso limitato alla sola consultazione dei dati anagrafici del locale e delle recensioni pubblicate, senza possibilità di effettuare operazioni di interazione o modifica; l'utente cliente, oltre alla visualizzazione dei dettagli, attiva funzionalità mirate all'invio di nuove prenotazioni e alla pubblicazione di recensioni e valutazioni, permettendo un'interazione diretta con l'offerta gastronomica; infine, l'utente gestore, quali la gestione completa del ciclo di vita delle prenotazioni ricevute, la facoltà di rispondere ufficialmente alle recensioni degli utenti e l'autorizzazione di modificare o eliminare le informazioni relative al proprio ristorante.

10.4 Pannello Gestione Prenotazioni (Ruolo Gestore e Cliente)

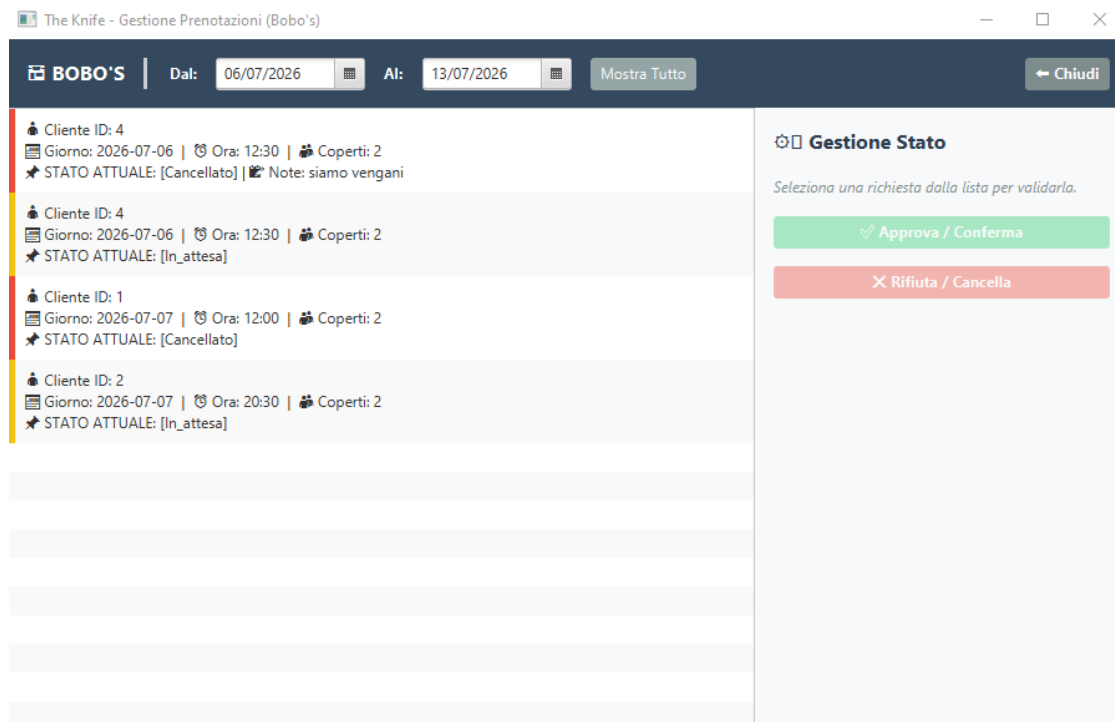


Figura 5.7: Prenotazioni ristorante (VisualizzaPrenotazioneRistoranteView)

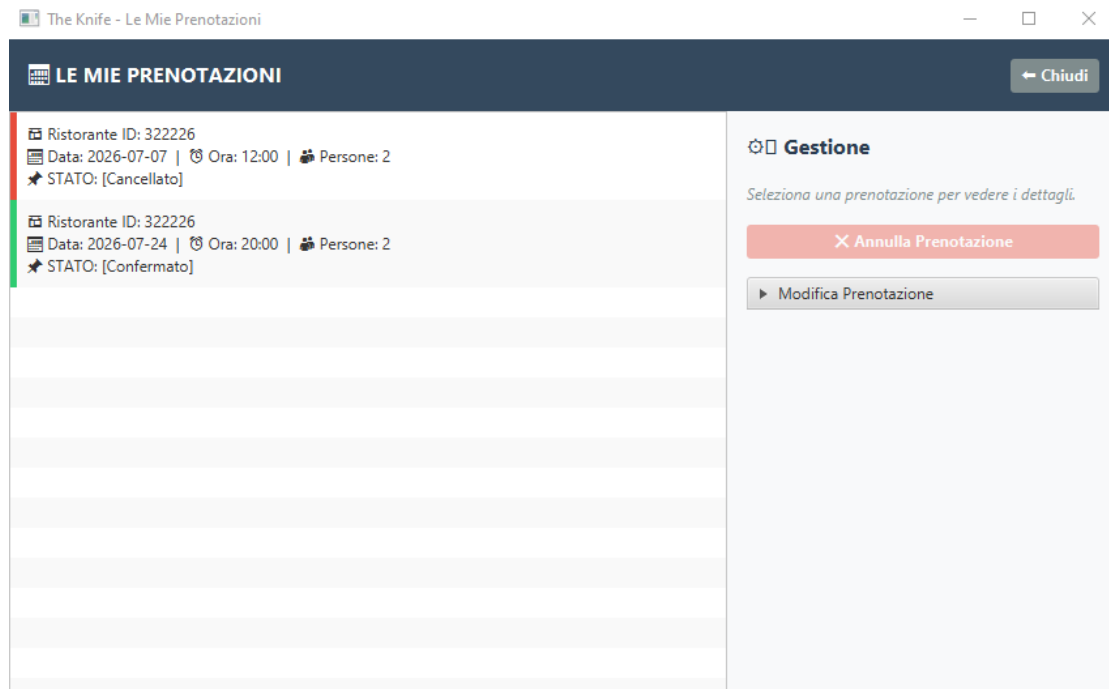


Figura 5.8: Prenotazioni cliente (*VisualizzaPrenotazioneClienteView*)

- Commento all'immagine: Il pannello di gestione delle prenotazioni, visibile sia dal lato gestore (*Figura 5.7*) che dal lato cliente (*Figura 5.8*), elenca cronologicamente le richieste con i relativi stati attuali (es. *In_attesa*, *Confermato*, *Cancellato*). Il gestore o il cliente possono interagire tramite i pulsanti dedicati per approvare, rifiutare o annullare le prenotazioni, modificando lo stato in tempo reale sul database PostgreSQL tramite il server RMI.

10.5 Pannello di Selezione del Ruolo (Admin)

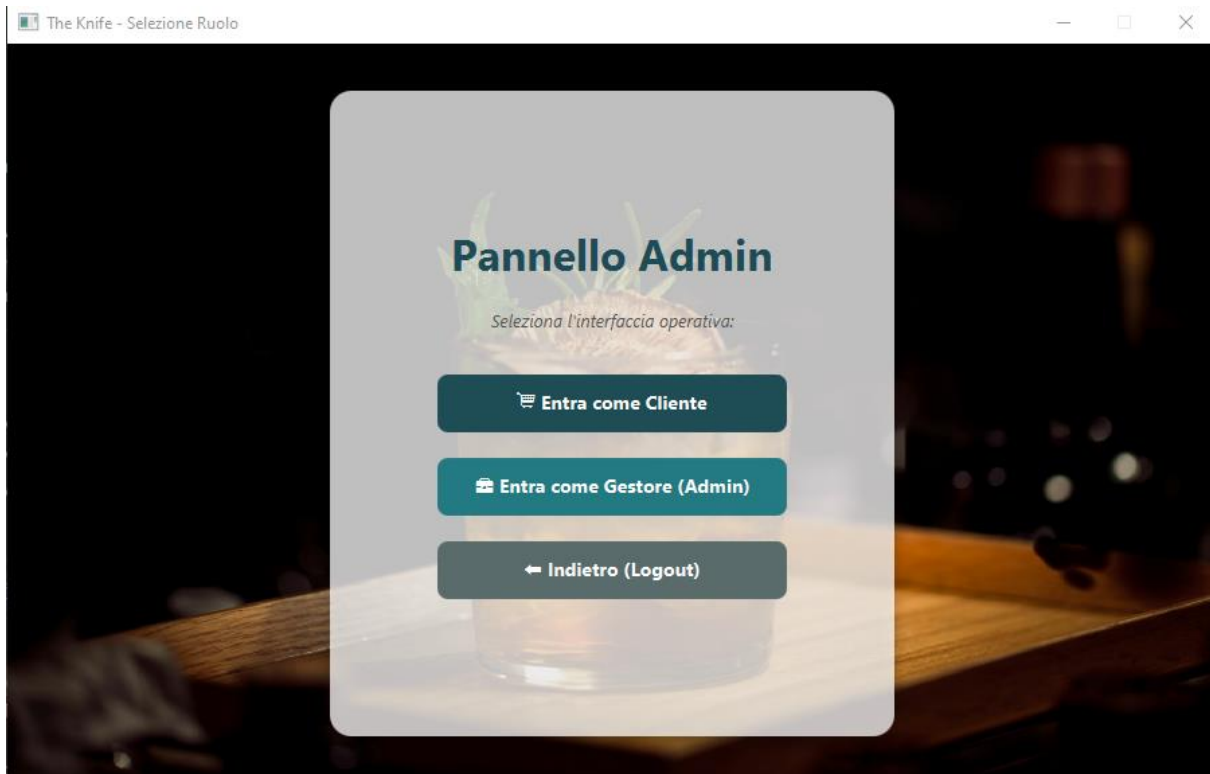


Figura 5.9: Home page per admin (SceltaRuoloAdminView)

- **Commento all'immagine:** Questa schermata intermedia viene presentata agli utenti dotati di privilegi amministrativi al momento dell'accesso. Il pannello consente di scegliere quale interfaccia operativa attivare tra "Entra come Cliente" e "Entra come Gestore", offrendo così la flessibilità di esplorare il sistema sia dal punto di vista del consumatore finale che da quello del gestore dei locali. È inoltre presente il pulsante di "Indietro (Logout)" per terminare la sessione corrente e ritornare alla schermata iniziale.